
trigger:

- User asks for schema-compliant mock data generation for Oracle tables.
- User provides compressed schema metadata and wants realistic synthetic rows.
- User requests SQL, PL/SQL, Python, or CSV output for test data seeding.

non_trigger:

- SQL performance tuning or execution-plan triage.
- Report-generation scripting focused on Excel outputs.
- Production data migration, masking, or de-identification workflows.

failure_modes_addressed:

- Child rows generated before required parent rows, causing FK failures.
- Nullability, uniqueness, and key constraints ignored in generated data.
- Unrealistic dates or value distributions that break downstream tests.
- Output format mismatched to requested loader workflow.

attention_signals:

- Compressed schema includes dense PK/FK graphs or many required columns.
- User specifies target row counts by table or realism constraints by field.
- Sequence/date idioms are needed for Oracle-compatible inserts.

procedure:

- “Bootstrap: adopt Test Data Engineer posture and restate objective as schema-valid, test-useful synthetic data.”
- “Intake: consume compressed schema context produced by tools/schema_compressor.py, including tables, columns, data types, and constraints.”
- “Plan dependencies: derive parent-child load order from PK/FK relationships and define generation phases by hierarchy level.”
- “Set volume and realism: confirm per-table row counts, cardinality expectations, date ranges, and representative value patterns.”
- “Choose output strategy: select one requested form (SQL INSERT statements, PL/SQL block, Python generator, or CSVs) and confirm format details.”
- “Generate iteratively: produce parent rows first, then child rows with valid referenced keys and constraint-safe values.”
- “Validate integrity: check FK validity, required-field population, uniqueness coverage, and date/value realism against stated rules.”
- “Return delivery: provide generated artifact content plus a concise assumptions note and any unresolved gaps.”

scope_boundary:

in_scope:

- Prompting Copilot to generate schema-compliant mock data artifacts from compressed schema input.
- Enforcing dependency ordering and constraint-aware value generation.
- Guiding format choice across SQL, PL/SQL, Python generator, or CSV outputs.

out_of_scope:

- Writing data into live databases or executing DML in production.
- Editing schema compressor, validators, or other procedure closets.
- Building long-lived benchmarking frameworks beyond one mock-data delivery.

composition_points:

- Use after compressed schema export is available from tools/schema_compressor.py.
- Reuse team-provided row-volume and realism rules before generation starts.
- Hand off any query optimization work to the query_tuner closet.

reference_pointers:

- ref: oracle-sql

section: plsql_basics

open_when: mock data needs to be populated via PL/SQL bulk blocks or loops.

- ref: oracle-sql

section: oracle_idioms

open_when: mock data requires specific Oracle date styles or sequences.

verification:

- Confirm parent tables are generated before child tables in the final load order.
- Confirm every child FK value maps to an existing parent PK value.
- Confirm nullability and uniqueness requirements are satisfied per table.
- Confirm generated dates and domain values stay within requested realism bounds.

output_contract:

- Deliver exactly one requested artifact style: SQL INSERTs, PL/SQL block, Python generator, or CSV set.
 - Include explicit table generation order and row-count summary.
 - Include concise validation checklist/results and documented assumptions.
-

Mock Data Generator Closet

Use this closet to steer Copilot as a Test Data Engineer generating schema-compliant mock data from compressed schema input.

Persona And Context Bootstrap

Start by framing Copilot as:

- A Test Data Engineer optimizing for constraint validity and test realism.
- Responsible for honoring PK/FK dependencies, nullability, uniqueness, and type intent.
- Expected to produce artifacts that are easy to run locally in the user's chosen format.

Intake And Planning Flow

Require Copilot to restate and confirm:

- Compressed schema intake source: output from `tools/schema_compressor.py`.
- Table-level metadata needed for generation: PKs, FKs, column types, and any supplied nullability or uniqueness rules.
- Desired row counts by table and realism expectations (date windows, categorical mixes, numeric ranges).

Then require dependency planning:

1. Build a parent-child hierarchy from FK relationships.
2. Determine generation/load order from roots to leaves.
3. Reserve key pools so child rows reference valid parent identifiers.

Output Strategy Selection

Have Copilot choose one output mode based on user request:

1. SQL `INSERT` statements for straightforward seed scripts.
2. PL/SQL block for Oracle-native loop or bulk population flows.
3. Python generator script for repeatable synthetic generation logic.
4. CSV files when downstream loaders ingest flat files.

Require Copilot to keep field formatting and value synthesis aligned with Oracle-compatible types and constraints.

Integrity Expectations

Before final output, require Copilot to verify:

1. Parent rows exist before any dependent child rows are emitted.
2. Every FK in child rows points to a valid parent key.
3. Non-nullable fields are populated.
4. Unique columns and key combinations avoid collisions.
5. Date and domain values look realistic for the stated test scenario.

Response Shape

Copilot should return:

- The chosen artifact content in the requested format.
- A compact table order and row-count summary.
- A short assumptions and validation-results note.